



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

AIML for Robotics

Module 2 – Sensors, Measurement, Model

Session Faculty:

Raja vadhana Prabhakar

The course owner and the authors of this course deck, Prof. Raja vadhana and Dr.Sangeetha, are gratefully acknowledging all authors who made their course materials freely available online.

☐ Introduction

- ☐ Types of Robots
- ☐ Mobile Robots – Use case of Programmable solutions

☐ Components of Programmable Robotic System

- ☐ Perception
- ☐ Representation
- ☐ Motion

☐ Localization

- ☐ Taxonomy
- ☐ Types

☐ Mapping

- ☐ Feature Based
- ☐ Grid Based

☐ Path Planning

- ☐ Graph Based
- ☐ Sample Based
- ☐ Evolutional Techniques

☐ Human – Robot Interaction

- ☐ Ethics
- ☐ Introduction to Visual Bot
- ☐ Introduction to Chat Bot

Sensor Measurement Model

Recap

1. Initial Belief
2. Control Update
3. Measurement Update

X_{t-1}	U_t	$P(X_t = \text{is_open})$	$P(X_t = \text{is_closed})$
is_open	push	1	0
is_closed	Push	0.8	0.2
is_open	do_nothing	1	0
is_closed	do_nothing	0	1

SENSOR MEASUREMENT MODEL : $P(Z_t | X_t)$

X_t	$P(Z_t = \text{sense_open})$	$P(Z_t = \text{sense_closed})$
is_open	0.6	0.4
is_closed	0.2	0.8

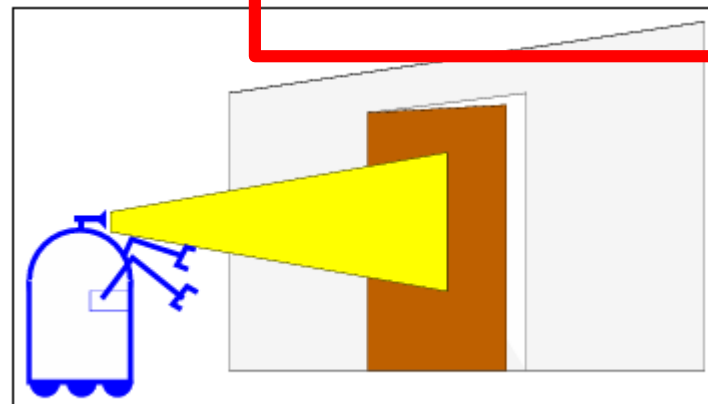
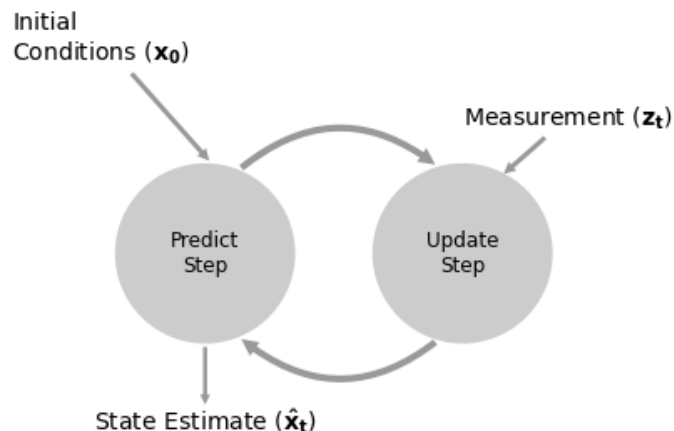
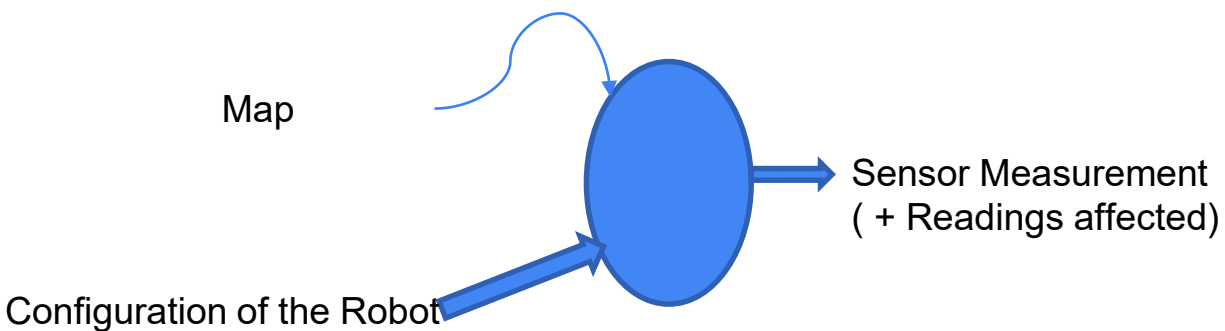


Figure 2.2 A mobile robot estimating the state of a door.

Inference :

The robot believes that with 0.983 probability the door is open

Sensor Model



Robot Localization: Hallway vs. Kitchen



The Bayesian Challenge

Priors: $P(\text{Hall}) = 0.7$, $P(\text{Kit}) = 0.3$

Observation: Microwave sound detected (Z).

Likelihoods:

- $P(\text{sound}|\text{Hall}) = 0.1$ (low)
- $P(\text{sound}|\text{Kit}) = 0.9$ (high)

Calculation & Belief

Step 1: Bayes Numerator

$$P(Z|\text{Kit})P(\text{Kit}) = 0.9 \times 0.3 = 0.27$$

Step 2: Normalization ($P(Z)$)

$$0.27 \text{ (Kit)} + 0.07 \text{ (Hall)} = 0.34$$

Final Belief $P(\text{Kit}|\text{sound}) = 0.794$

Result: Robot strongly believes it is in the Kitchen.

Bayesian Localization with Mixture Models



Noise Components

Hit: Normal detection (Gaussian)

Short: Unexpected dynamic obstacle

Max: Detection failure (Max range)

Rand: Unexplained random noise

$$P(z|x) = \sum w_i * P_i(z|x)$$

Mixture Calculation

Scenario: $z^* = 5\text{m}$, $z_{\text{obs}} = 4.7\text{m}$

Hit ($w=0.7$, $P=0.85$): 0.595

Short ($w=0.1$, $P=0.3$): 0.030

Rand ($w=0.1$, $P=0.05$): 0.005

Total $P(z|x, m) = 0.63$

Strongly supports pose hypothesis.

Beam Model

Deep Understanding of Measurement Models: $P(Z|X,m)$

CORE PROCESS: FROM SENSOR TO PROBABILITY

Finding $P(Z|X,m)$ is the process of evaluating how well an actual sensor reading (Z) matches what the robot expected to see (Z^*), given its assumed pose (X) and map (m).

- **Step 1 (Pose):** Robot assumes it is at location X (e.g., $x=6m$).
- **Step 2 (Map):** Map shows a wall at 10m.
- **Step 3 (Predict):** Expected $Z^* = 10 - 6 = 4m$.
- **Step 4 (Measure):** Sensor actually returns $Z = 3.8m$.
- **Step 5 (Evaluate):** Error $e = 3.8 - 4 = -0.2m$.

Is $P(Z|X,m)$ the Noise Distribution?

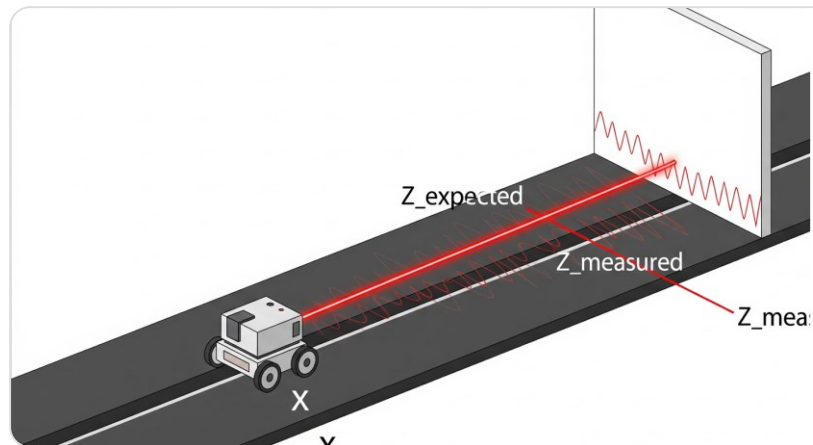
Not exactly. While the noise model defines the *shape* of the likelihood, the measurement model is the *application* of that noise relative to the map's geometry.

NUMERICAL EXAMPLE (GAUSSIAN HIT)

If sensor noise is Gaussian ($\sigma=0.2$) and error $e = -0.2$:

$$P(Z|X,m) = \eta * \exp(-(-0.2)^2 / (2 * 0.2^2)) = \eta * 0.606$$

A result of ~ 0.606 indicates a **high likelihood** that the robot is indeed at the hypothesized location.



Grounding Bayes' Rule: From Sound to Beam Models

THE SCENARIO

Conceptually replace abstract likelihoods with values computed by a physical beam model.

- **Hypothesis (X):** Robot is in the Hallway.
- **Map (m):** Hallway map predicts a wall at **4m** (Z^* or Z_{expected}).
- **Measurement (Z):** Laser sensor reports **3.9m** (Z_{measured}).

INFERENCE

If $Z = 3.9\text{m}$ is the evidence, the computed likelihood $P(Z|\text{Hallway}, m) \approx 0.882$ is the physically grounded value used as $P(Z|X)$ Bayes' Rule.

LIKELIHOOD STEPS (BEAM MODEL)

1. Ray Casting

Determines expected measurement Z^* if truly in hallway. $Z^* = 4\text{m}$.

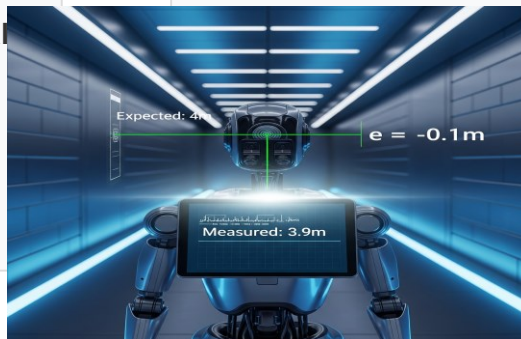
2. Error Calculation

Difference between actual and predicted.
 $e = 3.9\text{m} - 4\text{m} = -0.1\text{m}$.

3. Likelihood Computation (Mixture Model)

$P(Z|\text{Hall}, m) = \text{whitPhit} + \text{wshortPshort} + \dots$

Gaussian Hit ($\sigma=0.2$, $e=-0.1$):
 $P \approx 0.882$ (High Likelihood)



Beam Range Finder model - Algorithm

```

1:  Algorithm beam_range_finder_model( $z_t, x_t, m$ ):
2:       $q = 1$ 
3:      for  $k = 1$  to  $K$  do
4:          compute  $z_t^{k*}$  for the measurement  $z_t^k$  using ray casting
5:           $p = z_{hit} \cdot p_{hit}(z_t^k \mid x_t, m) + z_{short} \cdot p_{short}(z_t^k \mid x_t, m)$ 
6:               $+ z_{max} \cdot p_{max}(z_t^k \mid x_t, m) + z_{rand} \cdot p_{rand}(z_t^k \mid x_t, m)$ 
7:           $q = q \cdot p$ 
8:      return  $q$ 
    
```

$$p(z_t^k \mid x_t, m) = \begin{pmatrix} z_{hit} \\ z_{short} \\ z_{max} \\ z_{rand} \end{pmatrix}^T \cdot \begin{pmatrix} p_{hit}(z_t^k \mid x_t, m) \\ p_{short}(z_t^k \mid x_t, m) \\ p_{max}(z_t^k \mid x_t, m) \\ p_{rand}(z_t^k \mid x_t, m) \end{pmatrix}$$

Beam Model Summary

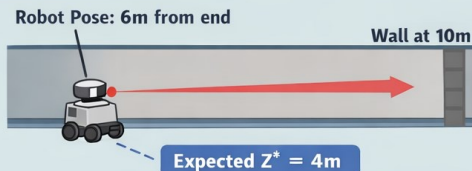
innovate

achieve

lead

Beam Model Integration for Sensor Measurement

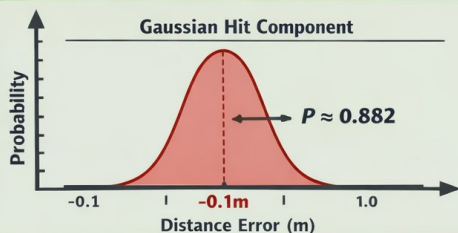
1. Ray Casting (Expected Measurement Z^*)



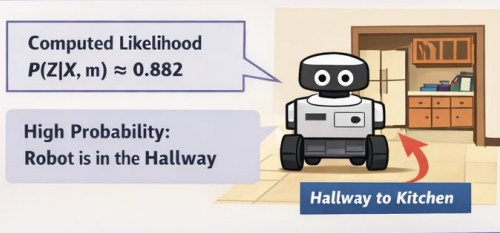
2. Error Calculation ($e = Z_{\text{measured}} - Z^*$)



3. Likelihood Computation (Mixture Model)



4. Localization Inference



$$P(z | x, m) = w_{\text{hit}} P_{\text{hit}} + w_{\text{short}} P_{\text{short}} + w_{\text{max}} P_{\text{max}} + w_{\text{rand}} P_{\text{rand}}$$

Mixture Model of Sensor Noise

Beam Model Summary

innovate

achieve

lead

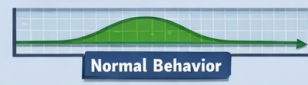
Beam Model Integration for Attack Detection

1. Baseline Prediction (Expected Behavior)



Normal Traffic Pattern:

- Logins: 9 AM – 5 PM
- Requests: < 100/min



Normal Behavior

2. Deviation Calculation

Observed Traffic:

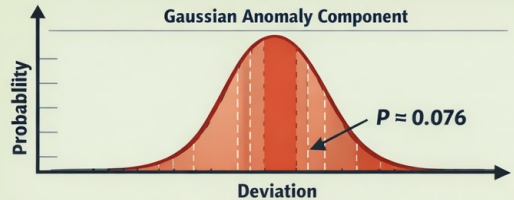
- Spikes at 2 AM
- Failed Logins: 20



Deviation = Observed - Expected

Deviation = Large

3. Likelihood Computation (Mixture Model)



$$P(\text{alert} | X) = w_{\text{hit}} P_{\text{hit}} + w_{\text{short}} P_{\text{short}} + w_{\text{max}} P_{\text{max}} + w_{\text{rand}} P_{\text{rand}}$$

Mixture Model of Threat Signals

4. Threat Assessment

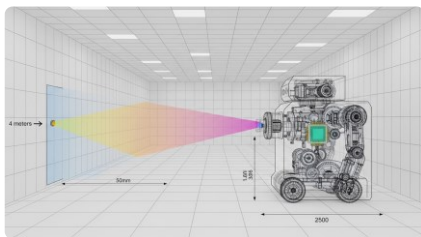
Computed Likelihood
 $P(\text{alert} | X) \approx 0.076$

High Likelihood:
Possible Attack Detected!



Working of Likelihood Model: Hallway vs. Kitchen

Hypothesis 1: Hallway



Prediction (Z^*): 4.0m

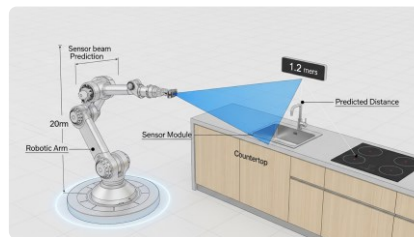
Likelihood: $P(Z=3.9|\text{Hall}) \approx 0.88$

High agreement with physical geometry.



Actual Measurement

Hypothesis 2: Kitchen



Prediction (Z^*): 1.2m

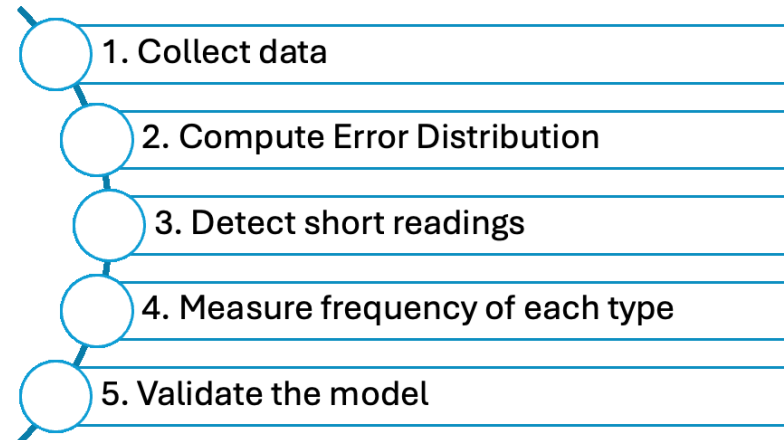
Likelihood: $P(Z=3.9|\text{Kit}) \approx 0.01$

Low agreement; hypothesis rejected.

- Observe the resulting density until it agrees with your experience.
- Maximizing the likelihood of reference data set Z - $p(Z | X, m, \Theta)$,

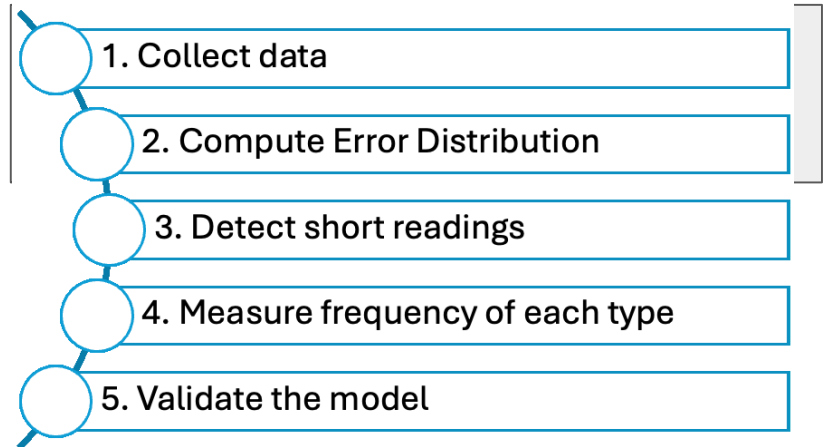
*Compare measured
distances vs expected
distances under
controlled conditions*

Data Driven learning



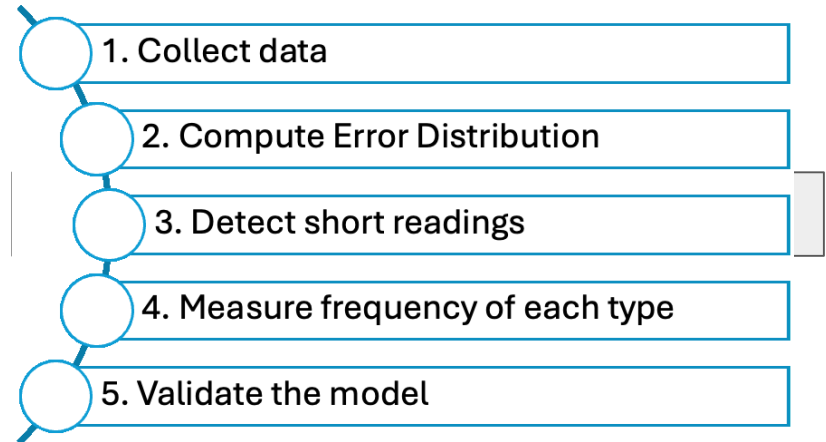
1. Get a set of samples $\{z_i, z_{\text{exp}}\}$
2. For each reading compute e_i

Data Driven learning



Data Driven learning

- Collect those $z < z_{\text{exp}}$ values and fit an **exponential distribution**
- Estimate the parameter λ_{short} using **Maximum Likelihood Estimation (MLE)**



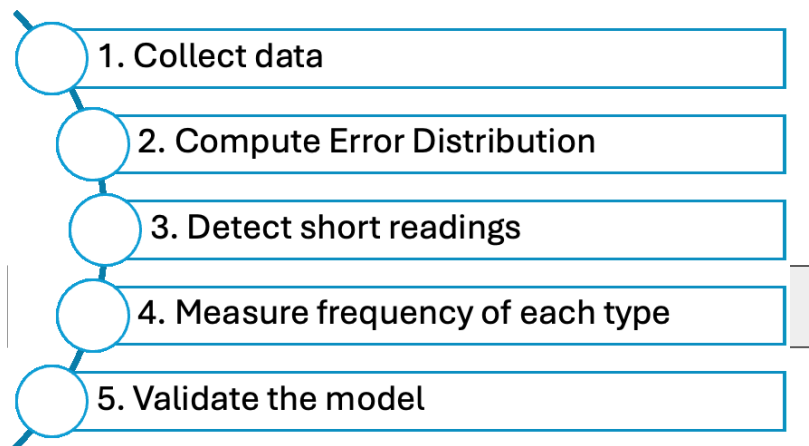
Learning Intrinsic parameters



Type	Condition	Count
Hit	$z \approx z_{\text{exp}}$ (within few σ)	N_{hit}
Short	$z < z_{\text{exp}} - \epsilon$	N_{short}
Max	$z = z_{\text{max}}$	N_{max}
Random	everything else	N_{rand}

Estimate mixture weight and normalize them so they sum to 1.

Data Driven learning

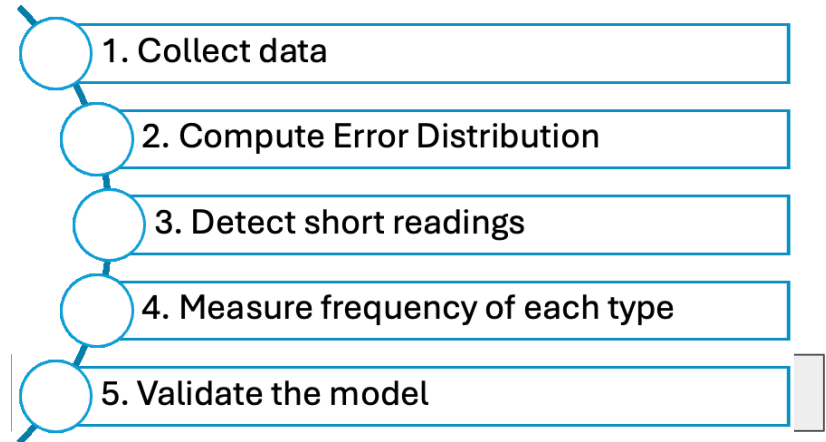


Reconstruct the full model . Then compare:

- predicted probabilities vs actual histogram of readings
- e.g., via log-likelihood or visual curve fitting.

If it matches well → your learned parameters are correct.

Data Driven learning



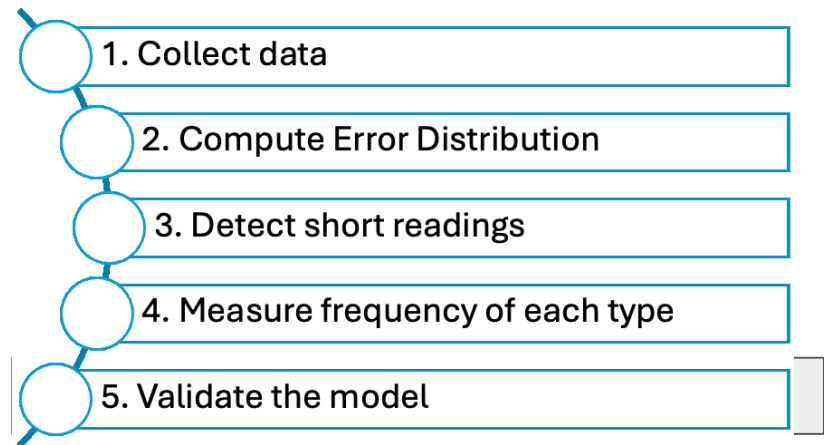
Learning Intrinsic parameters



Example :

Type	Count	Description
Hit readings around 19.8–20.2 m	760	Correct detections
Short readings (5–18 m)	110	Unexpected obstacles
Max range (30 m)	50	Failures to detect
Random spikes	80	Random noise

Data Driven learning



Learn Intrinsic Parameters



```
1: Algorithm learn_intrinsic_parameters( $Z, X, m$ ):
2:   repeat until convergence criterion satisfied
3:     for all  $z_i$  in  $Z$  do
4:        $\eta = [p_{\text{hit}}(z_i | x_i, m) + p_{\text{short}}(z_i | x_i, m)$   

5:          $+ p_{\text{max}}(z_i | x_i, m) + p_{\text{rand}}(z_i | x_i, m)]^{-1}$ 
6:       calculate  $z_i^*$ 
7:        $e_{i,\text{hit}} = \eta p_{\text{hit}}(z_i | x_i, m)$ 
8:        $e_{i,\text{short}} = \eta p_{\text{short}}(z_i | x_i, m)$ 
9:        $e_{i,\text{max}} = \eta p_{\text{max}}(z_i | x_i, m)$ 
10:       $e_{i,\text{rand}} = \eta p_{\text{rand}}(z_i | x_i, m)$ 
11:
12:       $z_{\text{hit}} = |Z|^{-1} \sum_i e_{i,\text{hit}}$ 
13:       $z_{\text{short}} = |Z|^{-1} \sum_i e_{i,\text{short}}$ 
14:       $z_{\text{max}} = |Z|^{-1} \sum_i e_{i,\text{max}}$ 
15:       $z_{\text{rand}} = |Z|^{-1} \sum_i e_{i,\text{rand}}$ 
16:
17:       $\sigma_{\text{hit}} = \sqrt{\frac{1}{\sum_i e_{i,\text{hit}}} \sum_i e_{i,\text{hit}} (z_i - z_i^*)^2}$ 
18:       $\lambda_{\text{short}} = \frac{\sum_i e_{i,\text{short}}}{\sum_i e_{i,\text{short}} z_i}$ 
19:
20:   return  $\Theta = \{z_{\text{hit}}, z_{\text{short}}, z_{\text{max}}, z_{\text{rand}}, \sigma_{\text{hit}}, \lambda_{\text{short}}\}$ 
```



Augmenting Beam Sensor Models: ML, DNN, and RL Integration

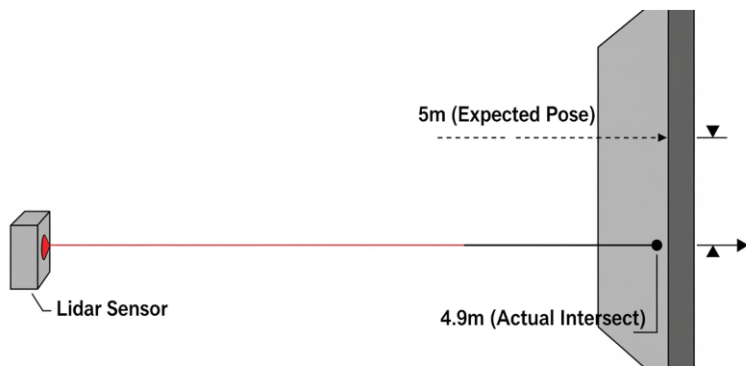
Substep / Parameter	AI/ML Approach	Usecase: Impact on Beam Model Accuracy
Noise Param (σ, λ) Intrinsic weights for beam probability density.	Classical ML (Expectation-Maximization) Iteratively fits Gaussian and Exponential parameters to historical sensor logs.	Indoor Office Navigation: Learns the specific noise profile of LiDAR sensors against glass walls vs. drywall, refining the 'random hit' weight to reduce phantom obstacles.
Beam Likelihood Field Spatial probability map for measurement z.	DNN (Deep Neural Networks) Learns non-linear likelihood distributions from raw pixel/point cloud data.	Dynamic Crowded Malls: A CNN-based model predicts "expected" beams in motion, allowing the bot to ignore beams hitting transient humans while focusing on static structural pillars.
Subsampling Rate Selecting subset of beams for efficiency.	RL (Reinforcement Learning) Policy-based agent selects which beams to process to maximize information gain.	UAV High-Speed Mapping: RL agent dynamically reduces beam density in uniform corridors to save CPU, but increases sampling at intersections to maintain pose confidence.

When Beam Models Outperform: Physics-Based Logic

Mobile Robot (Warehouse)

Scenario: Precise localization in a static warehouse with known geometry but zero visual landmarks (e.g., reflective metal racks).

- **Beam Model Fit:** Uses raw physics (ray-casting) to match expected vs. observed distances. No training required.
- **Failure of Others:** Feature models fail in repetitive aisles (perceptual aliasing). DNNs require massive labeled data for every layout change.



Software Agent (CyberBot)

Scenario: Monitoring internal network "distance fields" for policy violations in high-security, low-compute nodes.

- **Beam Model Fit:** Treats packet signatures as "beams" hitting security boundaries. Interpretable and extremely fast (500x vs trace).
- **Failure of Others:** Correlation models are too heavy for edge nodes. RL/AI methods introduce "black box" uncertainty in safety-critical protocols.

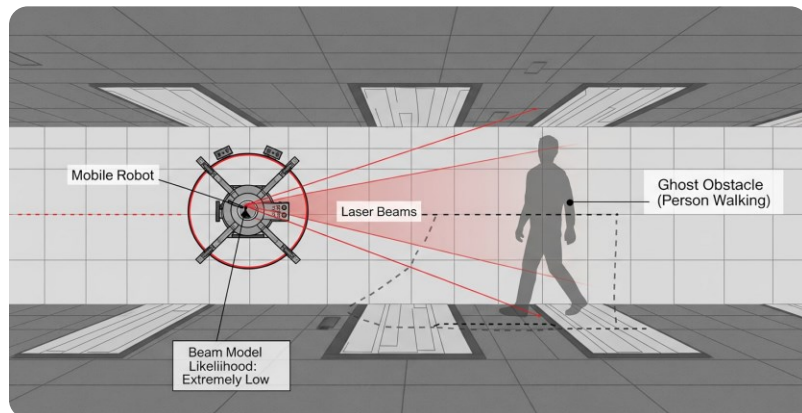
The Justification

Interpretability & Physics-First

Beam models win when geometry is fixed and causality is required. They bypass the need for data-heavy training while providing 100% explainable results.

Limitations of Beam-Based Sensor Models

The Dynamic Failure: Mobile Robots



Challenges

- Non-Smoothness
- Dynamic Sensitivity
- Computation

AIML Solution :

Likelihood Field (Endpoint) Model + ML

Instead of tracing the path, modern systems use **pre-calculated distance maps** that "smooth" the environment.

Deep Learning Integration: Recent models use DNNs to classify beams as "static" vs "dynamic," automatically filtering out human-induced noise before likelihood estimation.

Likelihood Fields - Map Matching

Likelihood Estimation: Raycasting vs. Endpoint Models

innovate

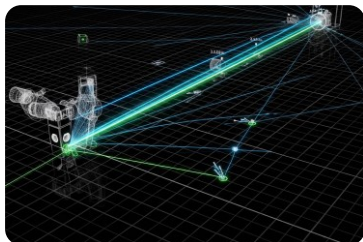
achieve

lead

Core Concept: The **Endpoint Model** (Likelihood Field) calculates likelihood $P(z|x)$ based on where a measurement ends, ignoring the path. It is a "snap-to-grid" approach compared to the journey-validated **Raycasting** model.

Raycasting (Trace Model)

"The Journey Matters": Validates the entire path from origin to hit. Asks if the journey was clear.



Complexity

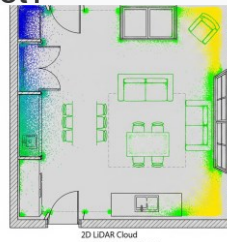
High ($O(N = \text{Distance})$)
Requires pixel-by-pixel
validation.

$$P(z|x) = P(\text{step}_i)$$

- **Mobile Bot:** Fragile to "ghost" map pixels.
- **Software Bot:** Trace every API hop (DNS TCP DB).
- **Logic:** High-fidelity, but computationally "melts" CPUs at scale.

Endpoint (Likelihood Field)

"The Result Matters": Only scores the final coordinate. Asks "how close is this result to the target?"



Complexity

Low ($O(N = 1)$)
Pre-calculated distance
lookup.

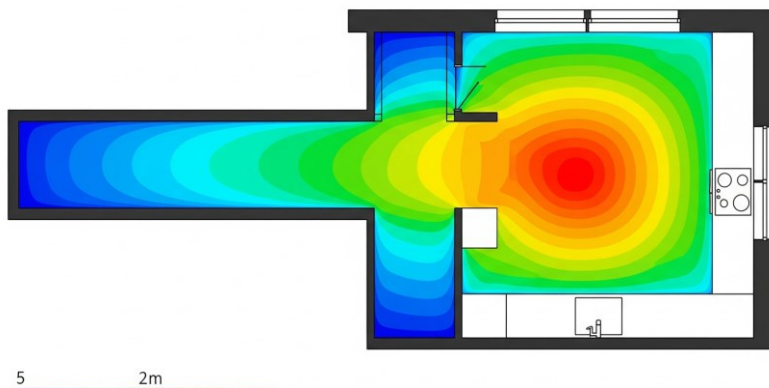
$$P(z|x) = \exp(-d/2)$$

- **Mobile Bot:** Robust to noise; "snaps" cloud to map.
- **Software Bot:** Outcome-based (Result vs. SLA).
- **Logic:** 500x faster; handles 100k+ points/sec.

Endpoint Likelihood Field: Approach

Step 1: Map Pre-computation

The environment is transformed into a distance field. Each cell stores the distance d to the nearest obstacle.



Step 2: Measurement Projection

The Mobile Bot projects its measurement ($Z=3.9m$) directly onto the map based on its current pose (X).

- **Hallway:** Endpoint lands 0.1m from wall.
- **Kitchen:** Endpoint lands 2.7m from wall.

Step 3: Likelihood Calculation

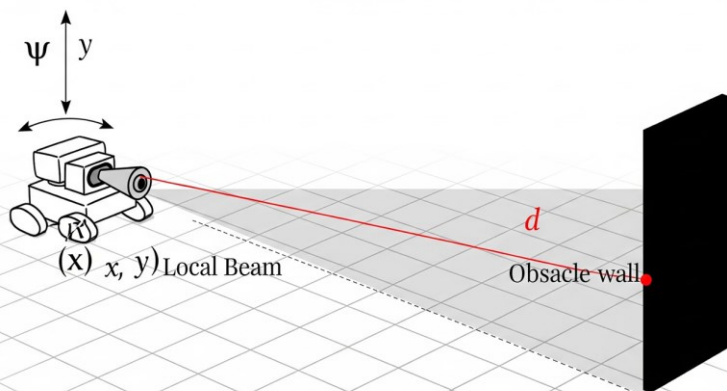
$$P(z|x, m) = \exp(-d^2 / 2\sigma^2)$$

Hallway Likelihood: ~ 0.88 (Match)

Kitchen Likelihood: ~ 0.01 (Reject)

Coordinate Transformation & Map Frame Alignment

Visualizing the Projection Problem



Endpoint $(x_e, y_e) = \text{Pose}(x, y) + \text{Rot}(\psi) * \text{Sensor}(z, \theta)$

Why Convert to Map Frame?

Sensor data is initially captured in the **Robot's Local Frame** (relative to its own center). Conversion to the **Global Map Frame** is critical because:

- Data Consistency
- Pose Estimation
- Likelihood Matching

Without the (x, y, ψ) transformation, the sensor readings would move with the robot, making it impossible to "stick" them to fixed obstacles.

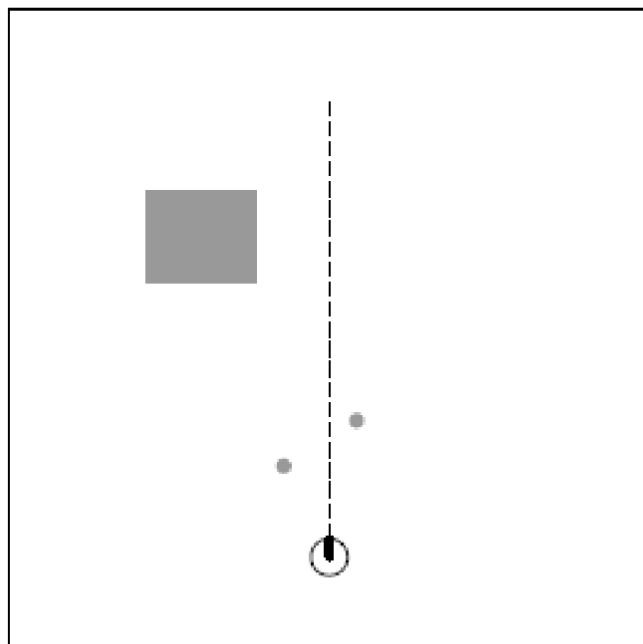
$$\begin{pmatrix} x_{z_t^k} \\ y_{z_t^k} \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x_{k,\text{sens}} \\ y_{k,\text{sens}} \end{pmatrix} + z_t^k \begin{pmatrix} \cos(\theta + \theta_{k,\text{sens}}) \\ \sin(\theta + \theta_{k,\text{sens}}) \end{pmatrix}$$

*** Only meaningful when the sensor detects an obstacle**

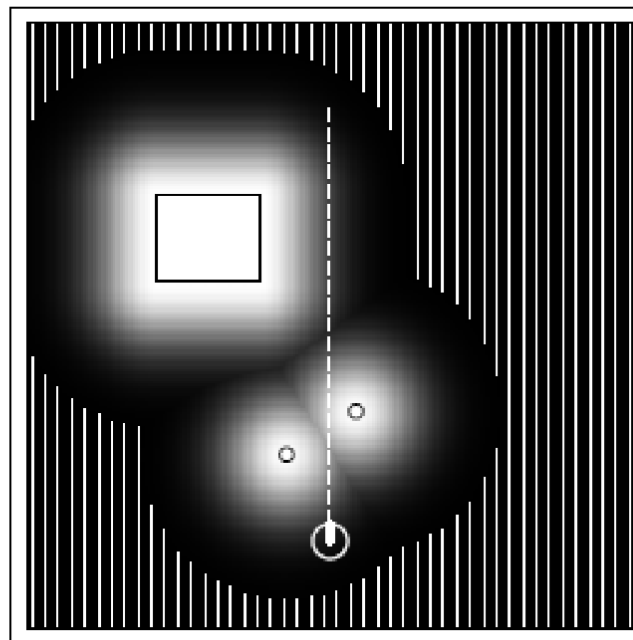
Likelihood field range finder



(a) example environment



(b) likelihood field

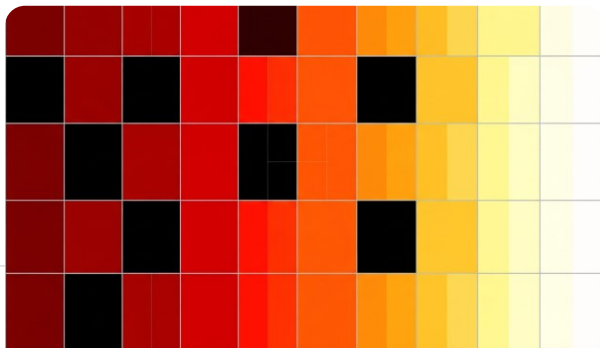


Likelihood field for (a)'s obstacle configuration

Mechanics of Endpoint Likelihood: Distance Fields & Projections

1. Distance Field & Grid Cells

The environment is discretized into a **Grid of Cells**. Each cell (i, j) in the **Distance Field** stores the Euclidean distance d to the nearest obstacle.



2. Measurement Projection

The sensor reading z_k (distance) at angle θ_k is projected from the robot's pose (x, y, ψ) to find the **Endpoint** (x_{end}, y_{end}) in the map frame:

- $x_{end} = x + z_k \cos(\psi + \theta_k)$
- $y_{end} = y + z_k \sin(\psi + \theta_k)$

Likelihood Equation

$$P(z|x, m) = \exp(-d^2 / 2 * \sigma^2)$$

Where d is the distance value of the cell where the endpoint lands.

Single Point Likelihood Calculation

1. Input Parameters

Robot Pose (x, y, ψ):

(10.0, 5.0, 0.0 rad)

Sensor Reading (z_k):

3.92m

Sensor Angle (θ_k):

0.523 rad (30°)

Assumed Sigma (σ):

0.1m

Note: $\theta=0.523$ rad is derived to yield the specific endpoint coordinates (12.1, 5.5) provided in the source table.

2. Step-by-Step Execution

A. Projection to Map Frame

$$x_e = 10 + 3.92 * \cos(0+0.523) \approx 12.1$$

$$y_e = 5 + 3.92 * \sin(0+0.523) \approx 5.5$$

B. Distance Field Lookup (d)

Cell (12.1, 5.5) stores distance **d = 0.08m**

C. Likelihood Score (P)

$$P = \exp(-(0.08^2) / (2 * 0.1^2))$$

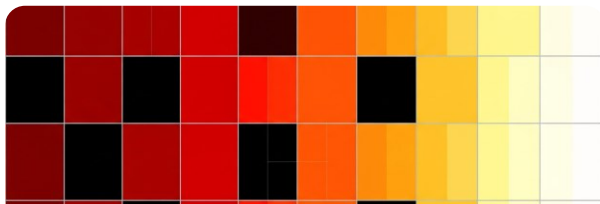
$$P = \exp(-0.0064 / 0.02) = \exp(-0.32)$$

$$P \approx 0.96 \text{ (High Match)}$$

Mechanics of Endpoint Likelihood: Distance Fields & Projections

1. Distance Field & Grid Cells

The environment is discretized into a **Grid of Cells**. Each cell (i, j) in the **Distance Field** stores the Euclidean distance d to the nearest obstacle.



2. Measurement Projection

The sensor reading z_k (distance) at angle θ_k is projected from the robot's pose (x, y, ψ) to find the **Endpoint** (x_{end} , y_{end}) in the map frame:

- $x_{end} = x + z_k \cos(\psi + \theta_k)$
- $y_{end} = y + z_k \sin(\psi + \theta_k)$

Likelihood Equation

$$P(z|x, m) = \exp(-d^2 / 2 \cdot \sigma^2)$$

Where d is the distance value of the cell where the endpoint lands.

3. 5-Point Cloud Consolidation

Point #	Sensor z_k	Endpoint (x_{end} , y_{end})	Field Dist (d)	Score (P)
1	3.92m	(12.1, 5.5)	0.08m	0.96 (High)
2	4.05m	(12.2, 5.7)	0.12m	0.91 (Match)
3	3.88m	(12.0, 5.4)	0.05m	0.98 (V. High)
4	2.10m	(10.5, 4.2)	2.40m	0.02 (Reject)
5	4.01m	(12.2, 5.6)	0.10m	0.94 (Match)

Likelihood field range finder - algorithm



Algorithm `likelihood_field_range_finder_model`(z_t, x_t, m):

$q = 1$

for all k do

if $z_t^k \neq z_{\max}$

$$x_{z_t^k} = x + x_{k,\text{sens}} \cos \theta - y_{k,\text{sens}} \sin \theta + z_t^k \cos(\theta + \theta_{k,\text{sens}})$$

$$y_{z_t^k} = y + y_{k,\text{sens}} \cos \theta + x_{k,\text{sens}} \sin \theta + z_t^k \sin(\theta + \theta_{k,\text{sens}})$$

$$\text{dist}^2 = \min_{x', y'} \left\{ (x_{z_t^k} - x')^2 + (y_{z_t^k} - y')^2 \mid \langle x', y' \rangle \text{ occupied in } m \right\}$$

$$q = q \cdot \left(z_{\text{hit}} \cdot \mathbf{prob}(\text{dist}^2, \sigma_{\text{hit}}^2) + \frac{z_{\text{random}}}{z_{\max}} \right)$$

return q

Likelihood field range finder - algorithm



Algorithm `likelihood_field_range_finder_model`(z_t, x_t, m):

$q = 1$

for all k do

if $z_t^k \neq z_{\max}$

$$x_{z_t^k} = x + x_{k,\text{sens}} \cos \theta - y_{k,\text{sens}} \sin \theta + z_t^k \cos(\theta + \theta_{k,\text{sens}})$$

$$y_{z_t^k} = y + y_{k,\text{sens}} \cos \theta + x_{k,\text{sens}} \sin \theta + z_t^k \sin(\theta + \theta_{k,\text{sens}})$$

$$\text{dist}^2 = \min_{x', y'} \left\{ (x_{z_t^k} - x')^2 + (y_{z_t^k} - y')^2 \mid \langle x', y' \rangle \text{ occupied in } m \right\}$$

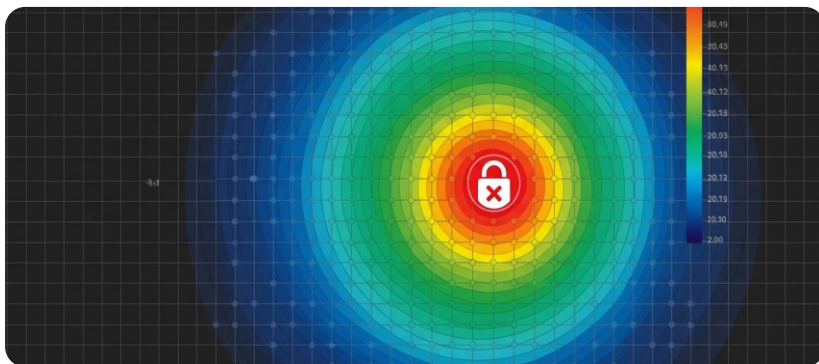
$$q = q \cdot \left(z_{\text{hit}} \cdot \mathbf{prob}(\text{dist}^2, \sigma_{\text{hit}}^2) + \frac{z_{\text{random}}}{z_{\max}} \right)$$

return q

Endpoint Likelihood Field: Cybersecurity Agent Monitoring

Step 1: Map Pre-computation

The network infrastructure is transformed into a "distance field" where each node or packet state stores the distance (d) to the nearest known malicious signature or policy violation.



Step 2: Measurement Projection

- The **Software Bot** projects its real-time telemetry (API calls, traffic volume) directly onto the pre-computed security map based on current state (X).
- **Healthy Flow**: Endpoint lands 0.1 units from safe baseline.
- **Anomalous Flow**: Endpoint lands 2.7 units from any safe zone.

Step 3: Likelihood Calculation

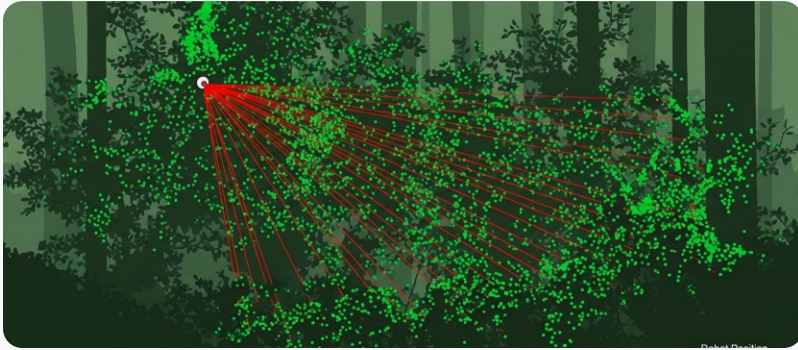
$$P(z|x, m) = \exp(-d^2 / 2\sigma^2)$$

Likelihood scores the final coordinate result against targets, enabling 500x faster validation than trace models.

Endpoint Likelihood: Optimal Scenarios

Mobile Robotics

Scenario: High-entropy, unstructured environments (e.g., dense foliage or shifting debris) where discrete features/landmarks are absent.



Software Agents

Scenario: High-velocity stream processing where the agent monitors non-deterministic arrival of independent API packets.





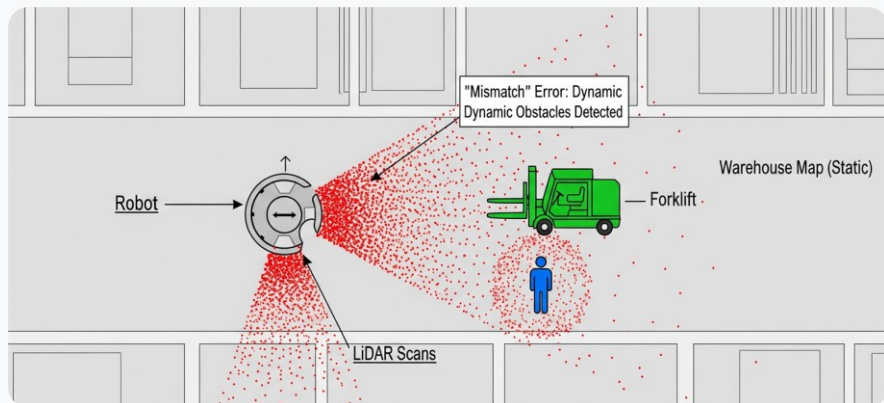
Augmenting Endpoint-Based Likelihood Models with AI

Substep/Parameter	AI/ML Approach	Distinct Endpoint Usecase
Intrinsic Noise Param (z_hit) Optimizing the standard deviation of the Gaussian endpoint noise.	Classical ML (EM) Iterative fitting of measurement noise distributions to historical endpoint logs.	Medical Imaging (Ultrasound SLAM): Learns the specific speckle noise characteristics of different tissue densities (e.g., bone vs. muscle) to dynamically sharpen the likelihood peaks at anatomical boundaries.
Measurement Decoding Extracting semantic meaning from endpoint clusters.	DNN (PointNet++) Directly processing raw endpoint clouds to predict obstacle classes.	Forestry Management Drones: A PointNet-based model classifies endpoints as "foliage" (soft noise) vs. "trunks" (hard landmarks), allowing the likelihood model to prioritize stable tree trunks for pose estimation.
Scan Matching Policy Adaptive search for the highest likelihood alignment.	Reinforcement Learning Policy-driven adjustment of search resolution based on pose uncertainty.	Autonomous Underwater Vehicles (AUV): The RL agent dynamically scales the likelihood search area: in flat sea floors, it subsamples endpoints to save power; in feature-rich coral reefs, it intensifies search to prevent drift.

Limitations & Challenges of Endpoint Likelihood Models

Failure Scenario: Dynamic Obstacles

In environments with moving entities (people, vehicles), endpoints hit transient objects not present in the static map, leading to localization drift.



CHALLENGES

- Ray-Casting Latency
- Sensitivity to Outliers
- Discretization Errors

AIML Solution

Hybrid DNN-Likelihood Fields

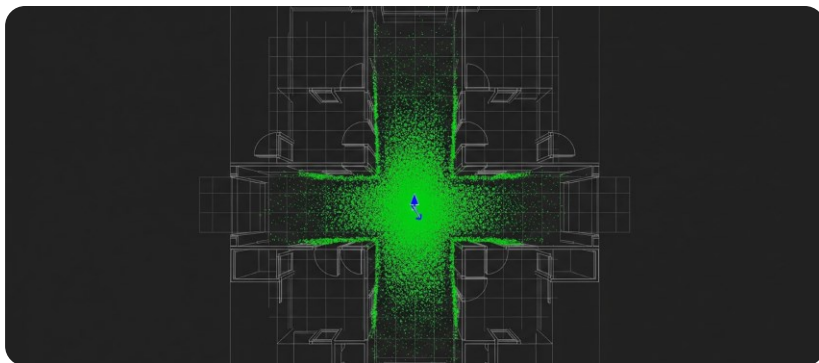
To classify endpoints before scoring..

Correlation Based Model

Correlation-Based Model

Step 1: Local Map Alignment

Scan-matching between current sensor data and the pre-existing occupancy map.



Step 2: Probabilistic Scoring

- **Hallway:** High overlap with parallel walls.
- **Kitchen:** Poor alignment; scan hits empty space.

Step 3: Likelihood Map Matching

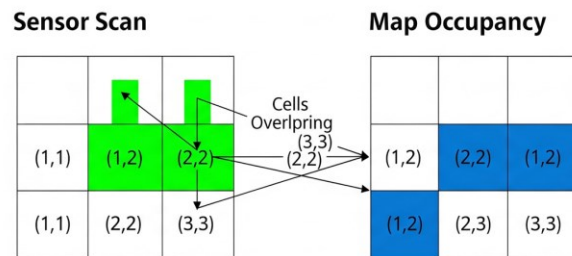
$$P(z|x, m) = \eta \cdot \sum (m_i \cap z_i)$$

Score measures pixel-by-pixel correspondence between sensed data and the map geometry.

Correlation

Example: Grid Map Alignment

Consider a 3x3 local grid where sensor data (z) and map data (m) are represented as binary values (1 for occupied, 0 for free).



Calculated Correlation Score

Correlation is determined by the intersection of occupied cells:

- **Intersection ($\cap$):** Count cells where both map and scan are '1'.
- **Normalization (η):** Scales the raw sum into a probability value.

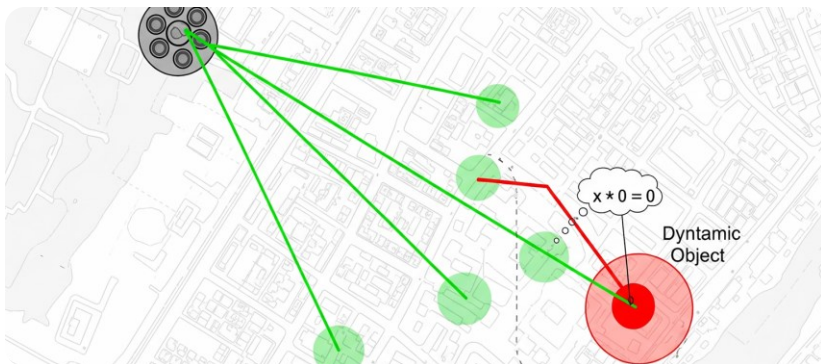
LIKELIHOOD MATCHING RESULT

$$P(z|x, m) = \eta \cdot \sum (m_i \cap z_i)$$

Resulting score quantifies the spatial correspondence between sensed geometry and stored maps.

The "Zero-Likelihood" Problem

In the product-of-probabilities approach, a single mismatch (e.g., Reading 3) where the sensor detects an obstacle in an unexpected free space results in an overlap score of 0.

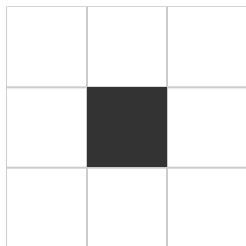


CRITICAL FAILURE STATE

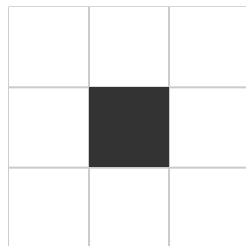
$$P(Z|X, M) = \dots \times 0 \times \dots = 0$$

Correlation-Based Model

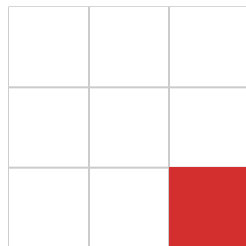
Step 1: Input Maps & Scans (3x3)



Map (M)



Scan A (Match)



Scan B (Mismatch)

Logic:

We evaluate the probability of the scan given the map at a specific pose. For each cell i :

- If Scan(i) matches Map(i), $P(z_i) = 0.9$
- If Scan(i) mismatches Map(i), $P(z_i) = 0.1$

Step 2: Joint Probability Update

Calculation for Scan B:

Total Cells = 9. Cells (1-8) match empty space. Cell 9 is a mismatch.

$$P(Z|X) = \prod P(z_i | x, m)$$

$$P(Z|X) = (0.9)^8 \times (0.1)^1$$

$$P(Z|X) = 0.4304 \times 0.1 = 0.043$$

Correlation-Based Model

Step 3: Score Aggregation

Comparing Scan A (Match) vs. Scan B (Mismatch) provides the relative likelihood of the robot's current pose.

Joint Probability Results:

Scan A (Perfect Match): $(0.9)^9 \approx 0.3874$

Scan B (1 Mismatch): $(0.9)^8 \times (0.1) \approx 0.0430$

Ratio of Likelihoods: $\sim 9:1$

Final Interpretation

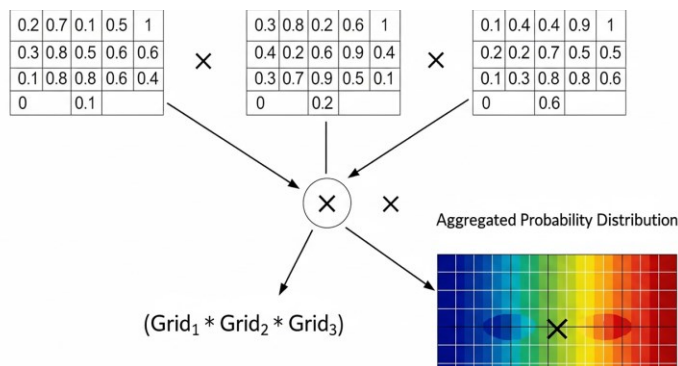
- **Graceful Degradation**
- **Localization Confidence**

Robust against dynamic obstacles like people moving or shifted furniture in the map.

Likelihood Aggregation

Probabilistic Aggregation

To compute the final belief, we aggregate the individual likelihoods across all grid cells using the standard product of probabilities algorithm.



Map Posterior

Peak of the aggregated map identifies the most likely position.

PRODUCT OF PROBABILITIES FORMULA

$$P(Z|X, M) = \prod \eta \cdot p(z_i | x, m_i)$$

The final product normalizes the individual cell correlations into a global likelihood score for the entire scan.

Map Matching and correlation measurement

innovate

achieve

lead

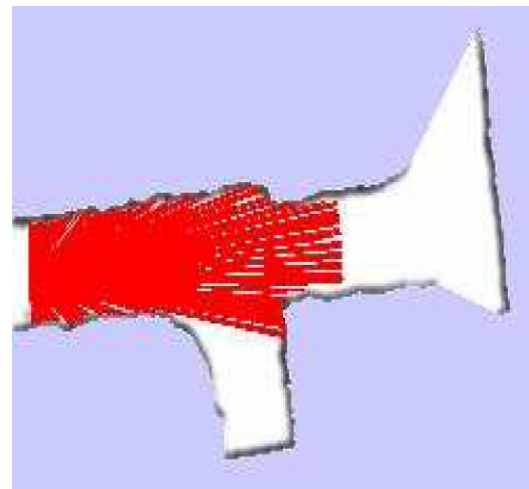
- compiles small numbers of consecutive scans into *local maps*, m_{local}
- Compare m and m_{local}

$$\rho_{m, m_{local}, x_t} = \frac{\sum_{x,y} (m_{x,y} - \bar{m}) \cdot (m_{x,y,local}(x_t) - \bar{m})}{\sqrt{\sum_{x,y} (m_{x,y} - \bar{m})^2 \sum_{x,y} (m_{x,y,local}(x_t) - \bar{m})^2}}$$

correlation ρ_{m, m_{local}, x_t} scales between 1

- **Probability of the local map conditioned on the global map m and the robot pose x_t**

$$p(m_{local} | x_t, m) = \max\{\rho_{m, m_{local}, x_t}, 0\}$$



Correlation-Based Sensor Measurement Modelling

Step 1: Spatio-Temporal Correlation

The software agent models cross-sensor dependencies, correlating packet rates and system calls across distributed nodes to distinguish coordinated attacks from local noise.



Step 2: Predictive State Matching

- **Expected State:** Sensor values remain within correlated bounds of the "Healthy Model".
- **Anomalous State:** Sudden loss of correlation between ingress traffic and CPU load suggests a hidden exfiltration process.

Agent applies a covariance matrix to validate multi-sensor consistency.

Step 3: Confidence Score

$$P(\text{breach} | z, C) = \eta \cdot P(z | C, M) P(C)$$

The joint probability of a breach given sensor observations (z) and the correlation model (C), identifying complex persistent threats.

When Correlation-Based Models Outperform?

Mobile Robotics

Scenario: Featureless Corridors



"Infinite Hallways"

Software Agents

Scenario: Zero-Day Log Analysis



When monitoring encrypted, unstructured telemetry for novel attacks, there are no known "signatures" (features) to match

Augmenting Correlation-Based Measurement Models with AI

Substep/Parameter	AI/ML Approach	Correlation-Specific AI Usecase
Scan Normalization (η) Optimizing the scaling factor for pixel correspondence.	Classical ML (Regression) Learns environment-specific normalization constants to prevent likelihood collapse.	Warehouse Logistics (AMRs): The ML agent analyzes historical occupancy grid drifts to adjust dynamically, ensuring that minor shelving shifts don't invalidate the entire map-matching score.
Dynamic Segmentation Filtering transient noise from the occupancy grid.	DNN (CNN / U-Net) Denoising sensor scans to separate static structures from moving objects.	Hospital Service Robots: A U-Net architecture processes 2D LiDAR "slices" to segment humans/gurneys as "noise," allowing the correlation model to match only the stable hallway geometry.
Search Step Optimization Tuning the "hill-climbing" scan matching search.	Reinforcement Learning Adaptive resolution policy based on environmental entropy.	Outdoor Last-Mile Delivery: The RL agent selects "coarse" search in featureless sidewalks to save power, but switches to "fine" high-resolution matching at complex street intersections.

Limitations and Challenges of Correlation-Based Models

Failure Case: Highly Dynamic Environments



POSE DRIFT & MAP CORRUPTION

Challenges

- Local Minima
- Symmetry Ambiguity
- Computational Cost

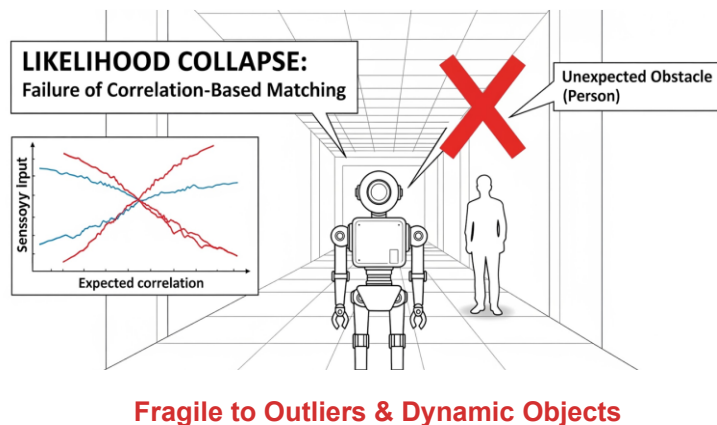
AIML Solution

Neural Likelihood Fields: Deep networks are trained to predict the correlation likelihood directly from raw scans.

From Correlation to Feature-Based Models

The Fragility of Correlation

Correlation relies on pixel-by-pixel overlap. A single mismatch (e.g., a person in a hallway) can cause the entire likelihood to collapse toward zero.



The Feature-Based Solution

- **Abstraction**
- **Robustness**
- **Efficiency**

Correspondence Scoring

$$P(z|x, m) = \prod P(f_i | x, m)$$

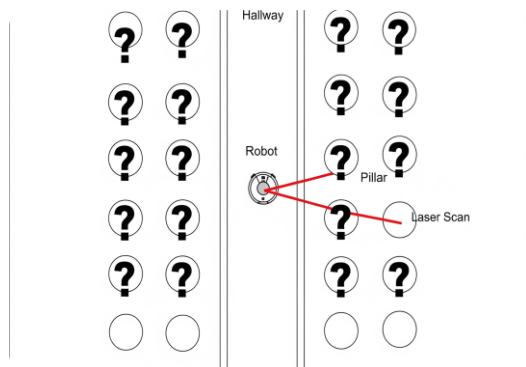
Likelihood is the joint probability of stable feature correspondences.

Stable & Physically Grounded

Feature Based Model

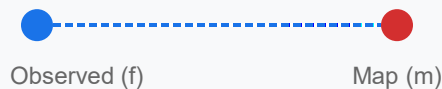
The Data Association Problem: Connecting Observations to Landmarks

The Problem



Observation (z_t) $\leftrightarrow$ Landmark (m_i) ?

The Solution: Distance



Validation Gate

$$d^2 = (f - m)^T \Sigma^{-1} (f - m)$$

Reject matches where $d > \chi^2$ threshold to ignore noise.

Result

Match Found!

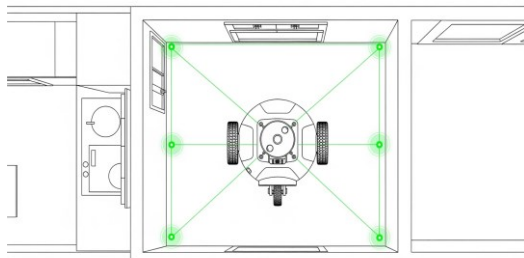
Identity confirmed; Bot updates its pose relative to 'Door_A'.

LOCALIZATION SUCCESS

Feature-Based Model

1. Feature Extraction

$$f(z_t) = \{f_t^1, f_t^2, \dots\} = \left\{ \begin{pmatrix} r_t^1 \\ \phi_t^1 \\ s_t^1 \end{pmatrix}, \begin{pmatrix} r_t^2 \\ \phi_t^2 \\ s_t^2 \end{pmatrix}, \dots \right\}$$



$$\mathbf{f} = \{\mathbf{x}: 2.1, \mathbf{y}: 0.5, \theta: 90^\circ\}$$

2. Data Association



Observed



Map (m)

Distance Calculation

$$d^2 = (\mathbf{f} - \mathbf{m})^T \Sigma^{-1} (\mathbf{f} - \mathbf{m})$$

If $d < \text{threshold}$, correspondence is accepted.

Threshold $\chi^2 = 3.84$ (Accepted)

3. Correspondence Scoring

$$p(f(z_t) | x_t, m) = \prod_i p(r_t^i, \phi_t^i, s_t^i | x_t, m)$$

Likelihood Model

$$\begin{aligned} P(\mathbf{z} | \mathbf{x}, \mathbf{m}) \\ = \prod P(\mathbf{f}_i | \mathbf{x}, \mathbf{m}) \end{aligned}$$

Aggregation

$$P(\mathbf{f}_1 | \mathbf{x}, \mathbf{m}) = 0.85$$

$$P(\mathbf{f}_2 | \mathbf{x}, \mathbf{m}) = 0.92$$

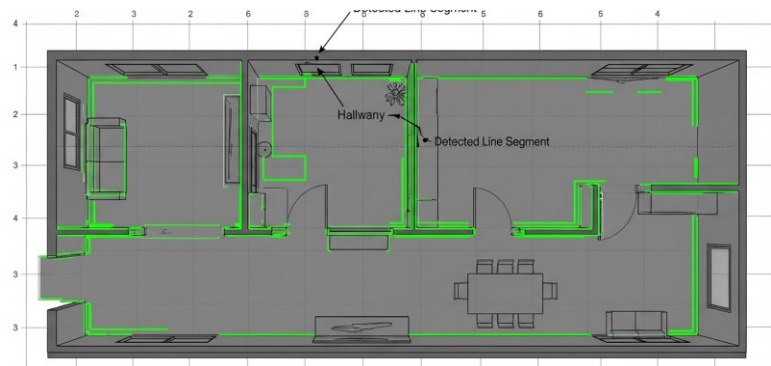
$$\prod P(\mathbf{f}_i) = 0.782$$

**TOTAL $P(\mathbf{Z} | \mathbf{X}, \mathbf{m}) = 0.782$
STABLE & ROBUST**

Feature-Based Model

Step 1: Feature Extraction

Lines, corners, or landmarks



Step 2: Landmark Matching

Extracted features are compared against a known map of features (M) using a data association algorithm.

- **Hallway:** Parallel lines match wall features.
- **Kitchen:** Features (corners) fail to find counterparts.

Step 3: Correspondence Scoring

$$P(z|x, m) = \prod P(f_i | x, m)$$

Likelihood is the joint probability of all feature correspondences being correct at pose X.

Feature based sensor model - algorithm



- Feature based maps – $m = \{m_1, m_2, \dots, m_N\}$
- Feature – signature, location coordinate $m_{i,x}$ and $m_{i,y}$

Measurement vector for landmark sensor

$$\begin{pmatrix} r_t^i \\ \phi_t^i \\ s_t^i \end{pmatrix} = \begin{pmatrix} \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2} \\ \text{atan2}(m_{j,y} - y, m_{j,x} - x) - \theta \\ s_j \end{pmatrix} + \begin{pmatrix} \varepsilon_{\sigma_r^2} \\ \varepsilon_{\sigma_\phi^2} \\ \varepsilon_{\sigma_s^2} \end{pmatrix}$$

Algorithm landmark_model_known_correspondence(f_t^i, c_t^i, x_t, m):

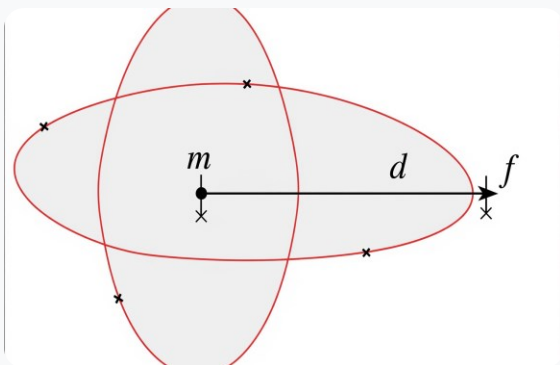
```
j = c_t^i
r_hat = sqrt((m_{j,x} - x)^2 + (m_{j,y} - y)^2)
phi_hat = atan2(m_{j,y} - y, m_{j,x} - x)
q = prob(r_t^i - r_hat, sigma_r^2) * prob(phi_t^i - phi_hat, sigma_phi^2) * prob(s_t^i - s_j, sigma_s^2)
return q
```

How the correspondence is computed?



The Mahalanobis Metric

Local correspondence is defined by the Mahalanobis distance (d^2), which scales the spatial error by the sensor's uncertainty covariance (Σ).



$$d^2 = (f - m)^T \Sigma^{-1} (f - m)$$

Gaussian Likelihood $P(f|x,m)$

The distance is transformed into a probability using a zero-centered Gaussian model. A small d^2 yields a high probability (match), while a large d^2 results in a near-zero likelihood (outlier).

Exponential Probability Model

$$P(f|x, m) \propto \exp(-\frac{1}{2} d^2)$$

Where d^2 is the Mahalanobis distance calculated in Step 4.

CONVERTS GEOMETRY TO PROBABILITY

How the correspondence is computed?



1. Input: 2D Features & Covariance

$f_1 = [2, 1], m_1 = [2, 1]$ (Match)

$f_2 = [5, 5], m_2 = [6, 5]$ (Near)

$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (Simple ID)

Step 4 Result: $d_1^2=0, d_2^2=1$

$$d^2 = (f-m)^T \Sigma^{-1} (f-m)$$

2. Probabilistic Aggregation

$$P(f_1|x, m) = e^{(-0)} = 1.00$$

$$P(f_2|x, m) = e^{(-0.5)} \approx 0.61$$

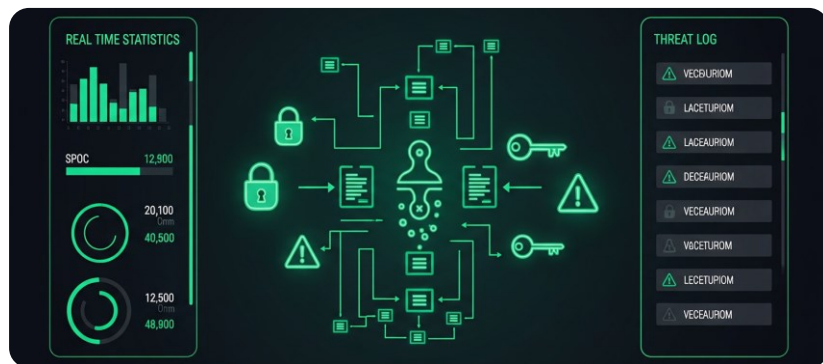
$$\prod P(f_i) = 0.61$$

FINAL $P(Z|X, m) = 0.61$ (STRONG MATCH)

Feature-Based Agent

Step 1: Signature & Behavior Extraction

The agent extracts high-level primitives (packet headers, API call sequences, or entropy shifts) from raw network traffic to simplify threat representation.



Step 2: Threat Pattern Matching

- **Safe State:** Features align with baseline "Golden Image" behaviors.
- **Attack State:** Extracted sequences (e.g., rapid lateral movement) trigger known exploit signatures.

Matching uses a data association algorithm against the global Threat Intel (M).

Step 3: Anomaly Scoring

$$P(\text{threat} | \mathbf{x}, \mathbf{m}) = \prod P(f_i | \mathbf{x}, \mathbf{m})$$

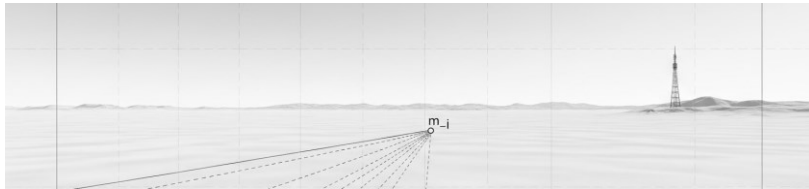
Confidence is the joint probability of all observed features indicating an active breach at node X.

Feature-Based Models: Superiority in Semantic & Sparse Environments

MOBILE ROBOTICS

Scenario: Long-Range Outdoor Landmark Navigation

Observation (z) $\leftrightarrow$ Landmark (ID)



SOFTWARE AGENTS

Scenario: Web Scraping Agent for E-commerce

- **Environment:** An agent extracting prices from a dynamic web page where the layout (DOM structure) changes constantly, but specific features (e.g., "Currency Symbol," "Add to Cart button") remain semantically distinct.

Feature-based models focus on **information-rich primitives**. They ignore the "noise" of empty space or irrelevant structural changes and robustness to global scale shifts.



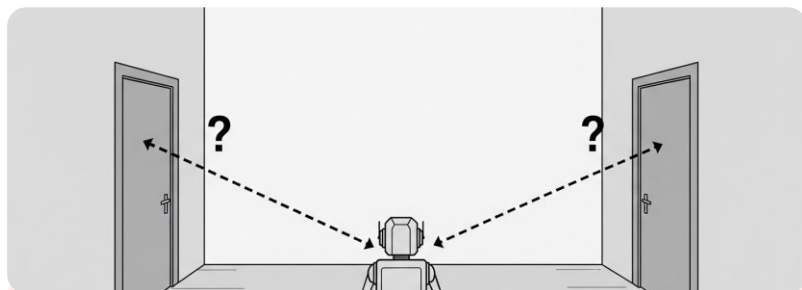
Augmenting Feature-Based Measurement Models with AI

Substep/Parameter	AI/ML Approach	Feature-Specific AI Usecase
Data Association (χ^2) Tuning Mahalanobis distance thresholds for landmark matching.	Classical ML (MLE) Learns optimal gating thresholds from sensor noise logs to reduce false positives.	Smart Office Navigation: The ML model adjusts validation gates based on historical sensor variance in narrow corridors, ensuring the robot doesn't misidentify similar-looking doorway corners.
Semantic Extraction Extracting high-level features like doors or signage.	DNN (YOLO / SSD) Robustly identifies and classifies landmark types in varying lighting conditions.	Retail Inventory Robots: A YOLO architecture identifies specific aisle markers and shelf headers as unique landmarks, providing a "semantic map" that is more robust than raw geometry.
Feature Selection Policy Choosing which landmarks to track in the state vector.	Reinforcement Learning Learns which features provide the highest information gain per compute cycle.	Outdoor Delivery Drones: The RL agent learns to ignore transient foliage and focus on stable rooflines or street signs, maximizing localization accuracy while minimizing data overhead.

Limits of Feature-Based Models: Perceptual Ambiguity & Scaling

LIMITATION: PERCEPTUAL ALIASING

Scenario: Indoor Navigation in Symmetric Environments



$\text{Observation}(z) \neq \text{Unique Landmark}(\text{ID})$

AIML Solution

DNN (YOLO/SSD): Extracts robust semantic primitives to differentiate "identical" geometry.



Hybrid Systems

Sensor Model Type	AI/ML Augmentation Role	Specific AI Method & Benefit
Beam & Likelihood Field Probabilistic ranging and distance matching.	- Learning intrinsic parameters (σ, λ, weights) from real sensor logs. - End-to-end noise modeling for complex scenes.	EM Algorithm / Regression: Fits mixture weights to history. DNN: Learns multi-modal noise in dynamic halls.
Correlation Based Map-matching and occupancy grid alignment.	- Adaptive scan-matching search space optimization. - Filtering dynamic objects from occupancy updates.	RL: Optimizes "hill-climbing" to find best state faster. CNN: Segments humans to prevent grid corruption.
Feature Based Landmark extraction (corners, doors, QR).	- Robust extraction of semantic primitives (SIFT, ORB). - Learning data association thresholds.	DNN / YOLO: Robustly identifies "Kitchen Door" vs "Hallway". ML: Learns Mahalanobis thresholds via MLE.

USE CASE: Warehouse Floor Automation

Scenario: A fleet of AMRs navigates reflective floors. **AI Benefit:** DNN-based likelihood fields outperform classical models in high-glare areas by learning material-specific patterns, while RL tunes motion parameters to adapt to varying floor slipperiness (wheel slip).

Summary



$$P(z | x, m) = w_{\text{hit}} p_{\text{hit}} + w_{\text{short}} p_{\text{short}} + w_{\text{max}} p_{\text{max}} + w_{\text{rand}} p_{\text{rand}}$$

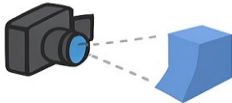
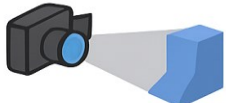
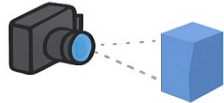
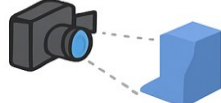
$$p_{\text{hit}} = \eta \cdot \exp\left(-\frac{(z - z^*)^2}{2\sigma^2}\right) \quad P(z | x, m) = \frac{1}{\sqrt{(2\pi)^n |\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{z} - \mathbf{z}^*)^T \Sigma^{-1} (\mathbf{z} - \mathbf{z}^*)\right)$$

$$p_{\text{short}} = \lambda e^{-\lambda z},$$

$$p_{\text{max}} = \begin{cases} 1 & z = z_{\text{max}} \\ 0 & \text{otherwise} \end{cases}$$

$$p_{\text{rand}} = \frac{1}{z_{\text{max}}}$$

Analogy: Different Ways to 'Understand a Photo'

Beam Model	Likelihood Field	Correlation Model	Feature-Based Model
			
Examine every pixel's brightness and geometry	Check how 'close' the picture is to a known shape	Compare overall similarity between images	Focus on key landmarks (eyes, corners, edges)
highly detailed but slow	fast but approximate	robust but less precise	efficient and abstract

$$P(z | x, m) = \eta \cdot \exp\left(-\frac{d(x, z)^2}{2\sigma^2}\right)$$

$$P(\mathbf{z} | x, m) = \prod_i \mathcal{N}(z_i; h_i(x, m), Q_i)$$

$$h_i(x, m) = \begin{bmatrix} r \\ \theta \end{bmatrix} = \begin{bmatrix} \sqrt{(x_m - x)^2 + (y_m - y)^2} \\ \tan^{-1} \frac{y_m - y}{x_m - x} \end{bmatrix}$$

65